

기록과 협업을 발판으로, 성장하는 개발자 장예지입니다.

Email: jj6129@naver.com

GitHub: [rubwan](https://github.com/rubwan)

Blog: [Velog](https://velog.io)



장예지 남개대학교 중국현대문학 석사 졸업 (2021.09 ~ 2024.06)

기능 기획부터 배포까지 지속적으로 기록하며 체계적인 개발을 합니다.
부트캠프와 인턴십을 통해 팀원과의 소통이 서비스 완성의 핵심임을 깨달았고,
인턴십 프로젝트 PM님과 함께 37개 개선사항 중 35개를 완료하며 협업의 이점을 체감했습니다.
매 프로젝트마다 WAVE와 LightHouse 도구를 활용해 웹 접근성을 점검하여, 현재까지 모든 프로젝트에서
Lighthouse 접근성 점수 90점 이상을 유지하고 있습니다. 누구나 서비스에 접근할 수 있는 웹 환경을 만들기
위해 고민하고 배우나가고 있습니다.

교육 및 실무 경험

2025.11 ~ 2026.05 **넷쟁이 사자처럼 프론트엔드 16 기 최우수 수료**

- HTML, CSS, JavaScript 기초부터 React, Next.js 까지 프론트엔드 개발자로서 익혀야 할 전반적인 핵심 기술을 습득했습니다.
6개월 교육 기간 동안 총 2번의 팀 프로젝트 경험이 있습니다.

2026.07.03 ~ 2026.07.29 **넷쟁이 사자처럼 로켓단 23 기 인턴십**

- KX 엔터테인먼트와의 연계 프로젝트로, 영상 업로드 시 시가 효과음을 추천하고 자막을 생성해주는 영상 편집 프로그램의 프론트엔드 전체 개발을 담당했습니다.

기술 스택

- Next.js
- React
- TypeScript
- Tailwind CSS
- TanStack Query
- Supabase

프로젝트

1. **예매의 정석** 사용 기술 : HTML, CSS, JavaScript

HTML, CSS, JavaScript를 사용하여 영화 선택부터 결제까지 실제 예매 사이트에 필요한 기능을 구현한 사이트입니다.
본 프로젝트에서 결제페이지를 개발하였습니다.

영화 정보 확인란

- 사용자가 선택한 영화 및 관람 시간 그리고 선택한 좌석 등의 정보들이 저장된 local Storage 에서 불러와 화면에 표시했습니다.

할인/포인트 탭 메뉴

- 사용자 포인트 및 카드번호 등의 할인 적용 수단 품을 구현. 정확한 할인 금액 적용을 위해 포인트/카드 유효성 검증 로직 구현하여, 검증 통과 후 할인 포인트는 총 합계 금액란에 출력되게 하였습니다.

영화 결제페이지 진입 시 카드 로직 구현

- 사용자가 올바른 조작 흐름 (영화 선택 → 좌석 선택)이 아닌 바로 결제페이지 url로 접근했을 시, local Storage 에 필수 정보(영화 또는 좌석 선택)가 없으면 정보를 입력하지 않은 페이지로 리디렉션하여 카드 로직 구현했습니다.

상태 유지 기능 구현

- 웹 UI의 활성 상태가 새로고침/뒤로가기/링크 공유 시 기본 탭으로 초기화되는 문제가 발생했습니다. 이를 해결하기 위해 URLSearchParams를 활용해 탭 페인 id를 URL에 저장하고 새로고침 시 활성화된 탭 상태를 자동으로 복원되도록 만들어 사용자 경험 개선 및 SEO 최적화했습니다.

2. 핵심 마켓 사용 기술 : Next.js, React, TypeScript, Supabase, TanStack Query

Next.js와 React를 기반으로, 소상공인의 감성 문구 제품을 한 곳에 모아 소비자가 회원으로서 상품 검색, 장바구니 담기, 구매 및 결제, 회원 정보 관리 등을 이용할 수 있도록 구현한 문구류 중심의 오픈마켓 사이트입니다.

본 프로젝트에서 사용자의 역할에 따라 소비자와 판매자의 마이페이지 기능을 구현했습니다. 소비자 마이페이지에서는 주문 상품 및 필요한 상품 관리를 담당하였고, 판매자 마이페이지에서는 상품 등록 및 주문 상품의 배송 관리를 담당했습니다.

주문 내역 및 상품 정보 조회

- 사용자의 주문 및 상품 정보를 Supabase 관계형 데이터 조회를 통해 연동하여, 상품명·이미지·가격·수량·주문 상태·카테고리 등의 데이터를 화면에 표시하였습니다. 또한 TypeScript 타입을 적용해 복잡한 중첩 데이터의 타입 안정성을 확보했습니다.

배송 상태 관리

- 주문 상품의 배송 상태를 조회하고 변경할 수 있도록 구현했습니다. Supabase update를 활용해 변경된 상태를 서버 데이터에 반영하였습니다. 상태 변경 로직과 UI를 컴포넌트 단위로 분리하여 관심사를 분리하고 유지보수성을 향상시켰습니다.

서버 상태 및 캐시 관리

- 사용자·주문 데이터를 TanStack Query로 관리하고, 데이터 수정 후 관련 Query를 invalidate하여 최신 서버 데이터를 다시 조회하도록 구현하였습니다. 서버 상태와 UI 상태를 분리하고 데이터 변경 이후 화면의 최신 상태를 안정적으로 유지하였습니다.

3. 오디오 톨킷 사용 기술 : Next.js, React, TypeScript, Zustand, TanStack Query

사용자가 효과음에 필요한 영상을 업로드하면 AI가 해당 영상의 음성을 분석해 추천된 효과음을 추가하고,자막 생성에 필요한 경우에는 음성을 텍스트로 변환해 자막까지 만들어주는 기능들을 지원하는 영상 편집 사이트입니다. 본 프로젝트에서는 오디오 톨킷 사이트의 전반적인 프론트엔드 업무 담당하였습니다.

서버 상태와 클라이언트 상태의 분리 관리

- 서버에 실제로 저장된 데이터(효과음 추천 목록, 자막 내용 등)는 TanStack Query로 관리하여 자동 요청, 캐싱, 진행 상태 풀링 등을 처리했습니다. 반면 아직 서버에 반영되지 않고 사용자가 화면에서만 조작하고 있는 임시 작업 상태는 Zustand로 별도 관리했습니다. 이렇게 분리함으로써 확정된 데이터와 임시로 편집 중인 데이터를 명확히 구분할 수 있었고, 불필요한 서버 요청을 최소화하도록 설계했습니다.

사용자 피드백 시스템 구축

- 데이터 생성부터 다운로드까지 전 과정에 toast 라이브러리를 활용한 즉각적인 피드백 기능을 추가했습니다. 또한 탭 이동이나 탭 닫기처럼 외물리기 어려운 동작을 시도할 때 확인 모달을 띄워, 사용자가 한 번 더 의사를 확인하고 예기치 못한 작업 유실을 방지할 수 있도록 했습니다. 특히 이 모달이 열려 있는 동안에는 키보드 포커스가 모달 밖으로 벗어나지 않도록 포커스 트랩을 적용하여 사용자 경험을 향상시켰습니다.

웹접근성 검증을 통한 시맨틱 마크업 개선

- WAVE Evaluation Tool과 Web Developer 브라우저 확장 프로그램을 활용해 실제 코드의 접근성을 검증했습니다. 각 기능에 알맞게 의미있는 태그(button, li 등)를 사용하여 HTML 구조를 정리함으로써 웹 표준을 준수하고자 했습니다. 또한 단순한 디자인 목적으로만 쓰인 아이콘에는 aria-hidden을 붙여 스크린 리더가 불필요한 정보를 읽지 않도록 처리하였습니다.

4. 중단어 참고 사용 기술 : Next.js, React, TypeScript, Supabase, TanStack Query, React-Hook-Form, Zod

중국어 단어 검색 및 저장 기능 이외에도, 사용자가 저장한 단어를 직접 활용하여 단어를 공부하는 기능들이 있으면 어떨까 라는 생각에 시작되어 개발을 하게 되었습니다. 중단어 참고는 저장한 단어를 직접 예문을 작성 및 수정이 가능한 기능을 제공합니다.

또한 모든 사용자를 고려하여 단어 검색과 예문 작성이 음성 인식으로도 가능하도록 개발하였습니다.

SSR 환경을 고려한 인증 및 접근 제어

- Next.js App Router 환경에서 Supabase Auth를 안정적으로 사용하기 위해 @supabase/ssr를 활용하여 서버와 클라이언트의 인증 상태를 관리했습니다. 또한 Proxy에서 사용자의 세션을 확인하여 마이페이지와 같이 인증이 필요한 경로에 대한 접근을 제어하고, 비밀번호 재설정 과정에서는 PASSWORD, RECOVERY 속성을 사용하여 정상적인 복구 흐름과 일반적인 페이지 접근을 구분했습니다.

Lighthouse 기반 웹 성능 개선

- Lighthouse를 활용하여 배포된 서비스의 성능을 측정하고 LCP와 Long Task 등 주요 성능 지표를 기준으로 병목 원인을 분석했습니다. 결과를 바탕으로 Next.js Image를 활용한 이미지 최적화, 주요 이미지의fetchPriority 및 preload 적용하여 성능을 개선했습니다.

브라우저 음성 인식 API를 활용한 음성 검색 및 예문 작성

- 브라우저에서 제공하는 Web Speech API의 Speech Recognition을 사용하여 음성을 텍스트로 변환하고, 변환된 결과를 기존 텍스트 입력 기능과 동일한 방식으로 작동되도록 구현했습니다. 음성 검색에서는 인식된 텍스트를 검색어로 전달하고, 예문 작성에서는 인식된 텍스트를 입력값에 반영하도록 구성했습니다. 검색과 예문 작성 모두 공통된 흐름으로 처리했습니다.